LoRAMBo: Fighting LoRA Memory Bottlenecks With Optimized Rank Selection

Liam Cawley
University of Michigan

Working note; corrected October 2026

Abstract

This note studies how to distribute LoRA ranks under a trainable-parameter budget. A positive-definite, layerwise quadratic metric reduces the approximation of a specified task update to a truncated singular value decomposition. The resulting allocation problem has an exact dynamic program; choosing the next rank by benefit per parameter is a heuristic when layer costs differ. We give a counterexample to the earlier greedy guarantee, retain the gradient and curvature-approximation terms in the local loss bound, and distinguish stationarity of fixed-rank factor training from global optimality. The accompanying numerical checks test these mathematical statements. No language-model or image-classification performance claim is made.

1 Correction and scope

Earlier versions included GLUE, WikiText-103, and ImageNet tables without verified experimental records. Those tables are withdrawn. They also combined results attributed to different allocation methods, and the GLUE comparisons used different adapter-parameter budgets. Neither matched-budget improvement nor reduced peak training memory was established.

The earlier theoretical treatment contained several errors: approximation of pretrained weights was conflated with approximation of a task update; a local loss bound omitted its first-order term; ordinary Lanczos was applied to a nonsymmetric operator; and a greedy knapsack argument was treated as exact in the absence of estimation noise. The corrections below specify which statements hold for the surrogate problem and which questions remain empirical.

2 The allocation problem

For a layer $W_i \in \mathbb{R}^{d_i \times k_i}$, LoRA learns an update $U_i V_i$ with $U_i \in \mathbb{R}^{d_i \times r_i}$ and $V_i \in \mathbb{R}^{r_i \times k_i}$ while retaining the pretrained weights [1]. The adapter has $r_i(d_i+k_i)$ stored parameters. With $c_i = d_i+k_i$, the budget is

$$\sum_{i=1}^{L} c_i r_i \le B, \qquad r_i \in \{0, \dots, \min(d_i, k_i)\}.$$

This counts adapter parameters. Peak training memory additionally includes the base model, activations, gradients, and optimizer state.

Let D_i be a specified target *update*, such as an update estimated in a pilot fit. A task-optimal update is generally unknown. Approximation of W_i itself would address compression of the pretrained model, a different objective. Adaptive allocation also has established precedents, including AdaLoRA [4]; rank allocation alone is not a novelty claim.

3 Weighted low-rank approximation

Let $G_i \in \mathbb{R}^{d_i \times d_i}$ and $A_i \in \mathbb{R}^{k_i \times k_i}$ be symmetric positive-definite matrices. Damped second-moment factors from K-FAC are one possible choice [3]. For an approximation error $E_i = D_i - U_i V_i$, define

$$q_i(E_i) = \operatorname{tr}(G_i E_i A_i E_i^\top) = \|G_i^{1/2} E_i A_i^{1/2}\|_F^2.$$

With column-major vectorization, this quadratic form has matrix $A_i \otimes G_i$. Positive damping makes the inverse square roots used below well defined.

Proposition 1. *Put $\widetilde{D}_i = G_i^{1/2} D_i A_i^{1/2}$ and let its singular values be $\sigma_{i1} \geq \cdots \geq \sigma_{im_i} \geq 0$, where $m_i = \min(d_i, k_i)$. Then*

$$\min_{\operatorname{rank}(X) \leq r} q_i(D_i - X) = \sum_{j>r} \sigma_{ij}^2.$$

One minimizer is $X = G_i^{-1/2} (\widetilde{D}_i)_r A_i^{-1/2}$, where $(\widetilde{D}_i)_r$ is a truncated SVD.

Proof. Multiplication by the invertible square roots preserves rank, so $X \mapsto G_i^{1/2} X A_i^{1/2}$ is a bijection between the feasible rank sets. Apply the Eckart–Young theorem in the transformed coordinates [2]. A minimizer need not be unique when singular values at the truncation boundary coincide. □

The result is exact for fixed D_i, A_i, G_i. Estimating these quantities during fine-tuning, and ignoring interactions between layers, introduces assumptions that the matrix approximation theorem does not settle.

3.1 Computing spectral scores

The squared singular values are eigenvalues of the symmetric PSD operator

$$S_i = A_i^{1/2} D_i^\top G_i D_i A_i^{1/2}.$$

Matrix-vector products with S_i can be supplied to a symmetric eigensolver. The product $A_i D_i^\top G_i D_i$ has the same eigenvalues by similarity, but is generally nonsymmetric in the Euclidean inner product. Ordinary symmetric Lanczos is therefore inappropriate for that product. Arbitrarily preconditioning the operator can change the spectrum being scored.

4 Allocating ranks

For the separable surrogate, minimizing the discarded spectral mass is equivalent to maximizing

$$\sum_i V_i(r_i), \qquad V_i(r) = \sum_{j=1}^{r} \sigma_{ij}^2, \qquad \sum_i c_i r_i \leq B.$$

The marginal values within each layer are nonincreasing. If every rank increment costs the same amount, repeatedly taking the largest available marginal value is optimal: the largest affordable number of values can be selected while respecting each layer's prefix order.

4.1 Unequal costs

For integer costs, an exact dynamic program is

$$F(i,b) = \max_{\substack{0\le r\le m_i \\ c_i r\le b}} \{F(i-1, b-c_i r) + V_i(r)\}, \qquad F(0,b) = 0.$$

Its running time depends on the numerical budget, so it is pseudopolynomial. For larger budgets, a benefit-per-cost greedy allocator is a practical alternative, provided each increment is checked against the remaining budget.

Example 1 (A counterexample to the earlier guarantee)**.** *Take two layers, each admitting one rank increment. Their costs are* 2 *and* 3*, their gains are* 3 *and* 4*, and the budget is* 3*. Greedy benefit per cost selects the first layer because* $3/2 > 4/3$*, leaving an unusable budget of* 1*. Its gain is* 3*, whereas choosing the second layer gives* 4*. The marginal gains are known exactly and diminishing returns holds trivially. Thus a bound asserting zero suboptimality when marginal-estimation error* $\delta = 0$ *is false for this rule. More strongly, give the second layer cost and gain* $B > 3$*, with budget* B*. Greedy still takes the first layer, attaining* $3/B$ *of the optimal gain. This ratio tends to zero; the unmodified rule has no constant approximation factor for arbitrary unequal costs.*

Online reallocation must account for *parameter cost*: reclaiming one rank from a narrow layer does not necessarily fund one rank in a wider layer. Recompute the remaining budget after removals and admit only affordable increments. Changes to ranks also require updating the factors and optimizer state; the allocation calculation by itself is not a training algorithm.

4.2 Continuous tail heuristics

If the discarded mass is approximated by $K_i r_i^{-(\alpha_i-1)}$ with $\alpha_i > 1$, a continuous interior optimum satisfies

$$r_i = \left(\frac{(\alpha_i - 1)K_i}{\lambda c_i}\right)^{1/\alpha_i},$$

with λ chosen to satisfy the budget. The square-root rule is the special case $\alpha_i = 2$. When exponents differ, the dependence on λ cannot be absorbed into a single common proportionality constant.

For $\sigma_{ij}^2 = C_i e^{-\gamma_i j}$, the infinite discrete tail is

$$\sum_{j=r_i+1}^{\infty} C_i e^{-\gamma_i j} = \frac{C_i}{e^{\gamma_i} - 1} e^{-\gamma_i r_i} = K_i e^{-\gamma_i r_i}.$$

The corresponding interior solution is $r_i = \gamma_i^{-1} \log(\gamma_i K_i/(\lambda c_i))$. Both rules require clipping to the rank bounds and a budget-feasible integer allocation. Their usefulness depends on how well the fitted tails describe the estimated update spectra.

5 What the quadratic metric says about loss

Consider changing one layer by E while holding every other layer fixed. Let g be that layer's loss gradient, H its Hessian block, and $C = A \otimes G$ the proposed PSD curvature surrogate. Assume

$\|H - C\|_{\mathrm{op}} \le \delta_C$ and that the Hessian is ρ-Lipschitz along the perturbation segment. Taylor's theorem gives

$$\begin{aligned}\mathcal{L}(\theta + E) - \mathcal{L}(\theta) \le \langle g, E\rangle + \tfrac{1}{2}\operatorname{tr}(GEAE^\top) \\ + \tfrac{\delta_C}{2}\|E\|_F^2 + \tfrac{\rho}{6}\|E\|_F^3.\end{aligned}$$

The gradient term vanishes only under an additional stationarity condition. K-FAC does not automatically upper-bound the true Hessian; the approximation error δ_C is needed. Mixed Hessian blocks enter when multiple layers are perturbed together, and must be bounded separately.

For example, with $\mathcal{L}(w) = w^2/2$, $w = 1$, and $E = 0.1$, the actual loss increase is 0.105. The quadratic term alone gives 0.005 even though the Hessian is represented exactly. Including the gradient term restores equality.

6 Fixed-rank training and stationarity

With ranks fixed, write the trainable factor parameters as $\theta = (U_1, V_1, \ldots, U_L, V_L)$ and the resulting loss as $f(\theta)$. Assume f is bounded below by f_{inf}, is β-smooth along the iterates, and has unbiased stochastic gradients with variance at most σ^2. For constant step size $\eta \le 1/\beta$, the standard SGD argument gives

$$\frac{1}{T}\sum_{t=0}^{T-1}\mathbb{E}\|\nabla f(\theta_t)\|^2 \le \frac{2(f(\theta_0) - f_{\mathrm{inf}})}{\eta T} + \beta\eta\sigma^2$$

[5]. This concerns stationarity in factor coordinates. Rank-constrained matrix updates form a nonlinear set, and the factorization is bilinear; they are not a fixed linear subspace onto which this gradient can simply be projected. The inequality gives neither a global loss-gap guarantee nor a comparison between rank-allocation policies.

7 Numerical checks and remaining evaluation

The accompanying `verify_allocation.py` reproduces the two-layer counterexample and the missing-gradient example. It also checks the weighted SVD identity and compares the dynamic program with exhaustive enumeration on small generated instances. These are numerical checks of the derivations and allocation code.

A model-performance evaluation would need identical base models, adapter placements, training data, optimization settings, and parameter budgets across allocation methods. It should retain per-seed results and measure peak memory and allocation overhead separately from adapter storage. Until those runs exist, there is no supported claim of improvement on GLUE, WikiText-103, or ImageNet.

References

[1] Edward J. Hu et al. *LoRA: Low-Rank Adaptation of Large Language Models*. ICLR, 2022. `https://arxiv.org/abs/2106.09685`.

[2] Carl Eckart and Gale Young. *The Approximation of One Matrix by Another of Lower Rank*. Psychometrika 1, 211–218, 1936.

[3] James Martens and Roger Grosse. *Optimizing Neural Networks with Kronecker-Factored Approximate Curvature*. ICML, 2015. https://proceedings.mlr.press/v37/martens15.html.

[4] Qingru Zhang et al. *Adaptive Budget Allocation for Parameter-Efficient Fine-Tuning*. ICLR, 2023. https://arxiv.org/abs/2303.10512.

[5] Saeed Ghadimi and Guanghui Lan. *Stochastic First- and Zeroth-Order Methods for Nonconvex Stochastic Programming*. SIAM Journal on Optimization 23(4), 2341–2368, 2013.